

HW4

讀檔

```
MLL_train <- read.delim("MLL_train.txt")
data <- MLL_train[-c(1:49), ]
```

1. Remove genes with name starting AFFX...(控制探針,共 49 個)

```
data <- MLL_train[-c(1:49), ]
```

2. Normalize data by each row(0~1)

```
data_norm <- t(apply(data[, 3:ncol(data)], 1, min_max_norm))
```

取得欄位名稱

```
col_name <- colnames(data_norm)
```

```
col_name # 觀察
```

清理欄位名

```
labels <- sub("_.*", "", col_name)
```

```
table(labels) # 觀察
```

Output:

```
> col_name # 觀察
[1] "ALL_1" "ALL_2" "ALL_3" "ALL_4" "ALL_5" "ALL_6" "ALL_7" "ALL_8" "ALL_9" "ALL_10" "ALL_11" "ALL_12" "ALL_13" "ALL_14"
[15] "ALL_15" "ALL_16" "ALL_17" "ALL_18" "ALL_19" "ALL_20" "MLL_1" "MLL_2" "MLL_3" "MLL_4" "MLL_5" "MLL_6" "MLL_7" "MLL_8"
[29] "MLL_9" "MLL_10" "MLL_11" "MLL_12" "MLL_13" "MLL_14" "MLL_15" "MLL_16" "MLL_17" "AML_1" "AML_2" "AML_4" "AML_5" "AML_6"
[43] "AML_7" "AML_8" "AML_10" "AML_11" "AML_12" "AML_14" "AML_15" "AML_16" "AML_17" "AML_18" "AML_19" "AML_20" "AML_21" "AML_22"
[57] "AML_23"
```

```
> table(labels) # 觀察
```

```
labels
```

```
ALL AML MLL
```

```
20 20 17
```

3. Find top 10 differentially expressed genes using t-test,

每組一次(該組 vs 其他),條件:mean(該組) > mean(其他)

- 三組各 10 個 → 合併成 30 selected genes

```
# 對每一 row 計算 t-test
count_t <- function(group_a, group_b){
  t_value <- numeric(nrow(group_a))
  for(i in 1:nrow(group_a)){
    t_value[[i]] <- t.test(group_a[i, ], group_b[i, ])$statistic
  }
  return(t_value)
}
```

ALL

```
# ALL
ALL_data <- data_norm[, which(labels == "ALL")] # 只有 ALL 的欄位資料
ALL_others_data <- data_norm[, -which(labels == "ALL")] # 除了 ALL 的欄位資料

ALL_mean <- apply(ALL_data, 1, mean) # 計算只有 ALL 欄位的每一列平均值
ALL_others_data_mean <- apply(ALL_others_data, 1, mean) # 計算除了 ALL 欄位的每一列平均值

# 只保留 mean(該組) > mean(其他)
ALL_data_filter <- ALL_data[which(ALL_mean > ALL_others_data_mean), ]
ALL_others_data_filter <- ALL_others_data[which(ALL_mean > ALL_others_data_mean), ]

# 計算 t-test
ALL_t <- count_t(ALL_data_filter, ALL_others_data_filter)

# 由大到小排列
ALL_t_sort <- order(ALL_t, decreasing = TRUE)

# 透過 order 回傳的前十個索引找到在 ALL_data_filter 的 gene
ALL_top_10_genes <- ALL_data_filter[ALL_t_sort[1:10], ]
ALL_top_10_genes
```

Output:

```
> ALL_top_10_genes
      ALL_1  ALL_2  ALL_3  ALL_4  ALL_5  ALL_6  ALL_7  ALL_8  ALL_9  ALL_10  ALL_11  ALL_12
11297 0.9848624 0.8891262 0.8534237 0.9567007 0.9597962 0.8664632 0.9791918 0.9448193 0.8670052 0.7645438 0.9116216 1.0000000
11119 0.9913581 0.8970998 0.7705845 0.9772651 0.9122111 0.9146329 1.0000000 0.8435916 0.9720321 0.8477198 0.8673236 0.9983391
2592 1.0000000 0.9222954 0.7615133 0.7619349 0.6664795 0.9285738 0.9384342 0.8335732 0.6910043 0.6044753 0.8955766 0.7679209
1737 0.5909829 0.7111378 0.8987824 0.7020958 0.5594575 0.7960181 0.7696413 0.6632961 1.0000000 0.6203240 0.6958946 0.6530149
11718 0.9342673 0.8340237 0.5828231 0.3935222 0.8706878 0.8359568 0.7736298 0.8358528 0.7987100 0.7049838 0.8048832 0.9153735
3768 0.8644278 0.6949892 0.9421570 1.0000000 0.8003958 0.8355479 0.9703693 0.9459753 0.9226441 0.9364695 0.9185549 0.8715198
6048 0.8250815 0.9329811 0.7929935 0.7175201 0.7175533 1.0000000 0.9853532 0.8912146 0.6793235 0.7792430 0.9423413 0.5577051
8165 0.7994577 0.7923819 0.8672277 0.8076994 0.8971798 0.7647789 1.0000000 0.8687943 0.8103312 0.7246733 0.8991704 0.8443561
5161 0.6189754 0.7534264 0.5321905 0.8095102 0.7389988 0.7674260 1.0000000 0.6331960 0.7578017 0.7883732 0.6716076 0.6358384
9882 0.6725414 0.6885509 1.0000000 0.7322728 0.7019789 0.7488396 0.8432285 0.7856280 0.8065272 0.6410734 0.7632969 0.6240681

      ALL_13  ALL_14  ALL_15  ALL_16  ALL_17  ALL_18  ALL_19  ALL_20
11297 0.7859925 0.8815295 0.9189933 0.9228019 0.76031473 0.9742810 0.9116264 0.6682369
11119 0.8851030 0.9061130 0.8939008 0.9476817 0.83396565 0.9431259 0.8114289 0.8808635
2592 0.6844902 0.8136074 0.6945411 0.6986897 0.68762608 0.8630808 0.6292300 0.6496663
1737 0.6742678 0.7694975 0.8195295 0.5909157 0.44321842 0.3799067 0.8209365 0.4637983
11718 0.6510945 0.9370876 0.8707101 0.9082491 0.75849299 0.5310410 0.4696286 1.0000000
3768 0.6881296 0.7200940 0.8895080 0.7470677 0.32125270 0.3885569 0.8318823 0.5135899
6048 0.6514529 0.9261236 0.4820085 0.5627891 0.05025188 0.6425075 0.6915221 0.6087870
8165 0.7110242 0.8760763 0.7944664 0.7739272 0.77078399 0.7676779 0.7864793 0.9026961
5161 0.9066447 0.7079520 0.8781760 0.9321081 0.37326767 0.6685788 0.8258061 0.4285675
9882 0.6337867 0.7734488 0.6945669 0.7149539 0.76601330 0.4404668 0.7090224 0.6279479
```

MLL

```
## 以下同理

# MLL
MLL_data <- data_norm[, which(labels == "MLL")]
MLL_others_data <- data_norm[, -which(labels == "MLL")]

MLL_mean <- apply(MLL_data, 1, mean) # 計算只有 MLL 欄位的每一列平均值
MLL_others_data_mean <- apply(MLL_others_data, 1, mean) # 計算除了 MLL 欄位的每一列平均值

# 只保留 mean(該組) > mean(其他)
MLL_data_filter <- MLL_data[which(MLL_mean > MLL_others_data_mean), ]
MLL_others_data_filter <- MLL_others_data[which(MLL_mean > MLL_others_data_mean), ]

MLL_t <- count_t(MLL_data_filter, MLL_others_data_filter)
MLL_t_sort <- order(MLL_t, decreasing = TRUE)

MLL_top_10_genes <- MLL_data_filter[MLL_t_sort[1:10], ]
MLL_top_10_genes
```

Output:

```
> MLL_top_10_genes
      MLL_1  MLL_2  MLL_3  MLL_4  MLL_5  MLL_6  MLL_7  MLL_8  MLL_9  MLL_10  MLL_11  MLL_12
7136 0.4278384 0.4200687 0.5689120 0.4682816 1.0000000 0.3325537 0.5198177 0.5869349 0.5110997 0.5164847 0.3347409 0.5177614
7930 0.7108589 0.6565026 0.7106757 0.8535774 1.0000000 0.8102612 0.6077619 0.5793913 0.6977712 0.7919025 0.8252975 0.7383112
2770 0.7865345 0.5431565 0.6756439 1.0000000 0.8634328 0.6630745 0.4060664 0.8093961 0.7097859 0.7155665 0.8810277 0.6962394
11643 0.8819003 0.8798990 0.9035411 0.7143826 0.8939032 0.9364203 0.9614462 0.9453761 0.9756892 0.8309684 1.0000000 0.8088625
1176 0.9004642 0.8291552 0.8944328 0.5349615 0.9134605 0.8779662 0.9134927 0.9025591 0.9059373 0.8412041 1.0000000 0.7627317
7170 0.8105482 0.7529724 0.9168918 0.9606659 0.7465141 0.8882806 0.8012196 1.0000000 0.9865598 0.9447637 0.9887736 0.8012932
10797 0.6562308 0.1006453 0.7280646 0.4643650 1.0000000 0.7529405 0.6521263 0.6995477 0.6543836 0.8713746 0.8276972 0.6898521
11368 0.7134995 0.7856813 0.6502153 0.3002523 0.4242359 0.8218419 0.8208518 0.7561282 0.7850668 1.0000000 0.4644174 0.5697278
8050 0.9401580 0.5649443 0.9492317 0.7194503 0.6736456 0.8027675 0.9709864 1.0000000 0.9078195 0.9030472 0.7574401 0.7058321
5591 0.8135583 0.6751563 0.8061541 1.0000000 0.6923172 0.7943863 0.8663445 0.7252219 0.5866740 0.5221802 0.6655601 0.8213311
      MLL_13  MLL_14  MLL_15  MLL_16  MLL_17
7136 0.4547034 0.5894780 0.4582842 0.5308940 0.32187381
7930 0.6794542 0.8343214 0.7270287 0.8116695 0.81908760
2770 0.8053476 0.4028579 0.7427233 0.8363210 0.01005141
11643 0.8214476 0.8899054 0.8939719 0.9134885 0.82695932
1176 0.8654825 0.8698611 0.9008846 0.9308955 0.88696144
7170 0.8560272 0.8277007 0.8128652 0.9730384 0.74338578
10797 0.8632446 0.9020847 0.7564247 0.7460246 0.66552470
11368 0.6867115 0.8881910 0.4124269 0.6250058 0.54633907
8050 0.9018394 0.8515757 0.7116483 0.9289865 0.54009919
5591 0.7122162 0.5800351 0.9694481 0.7929468 0.57259303
```

AML

```
# AML
AML_data <- data_norm[, which(labels == "AML")]
AML_others_data <- data_norm[, -which(labels == "AML")]

AML_mean <- apply(AML_data, 1, mean) # 計算只有 MLL 欄位的每一列平均值
AML_others_data_mean <- apply(AML_others_data, 1, mean) # 計算除了 MLL 欄位的每一列平均值

# 只保留 mean(該組) > mean(其他)
AML_data_filter <- AML_data[which(AML_mean > AML_others_data_mean), ]
AML_others_data_filter <- AML_others_data[which(AML_mean > AML_others_data_mean), ]

AML_t <- count_t(AML_data_filter, AML_others_data_filter)
AML_t_sort <- order(AML_t, decreasing = TRUE)

AML_top_10_genes <- AML_data_filter[AML_t_sort[1:10], ]
AML_top_10_genes
```

Output:

```
> AML_top_10_genes
      AML_1  AML_2  AML_4  AML_5  AML_6  AML_7  AML_8  AML_10  AML_11  AML_12  AML_14  AML_15
7407 0.7518352 0.9286722 0.7803987 0.6530851 0.7064778 0.7677808 0.8204811 0.8150803 0.71955493 0.8041279 0.9445423 0.6685531
6520 0.8349595 0.7009658 0.6652660 0.8030268 0.8002459 0.6268859 0.6237721 0.6176855 0.60287468 0.6098937 0.8395679 0.7168130
8468 0.8092866 0.8913676 0.8869210 0.5762715 0.6496942 0.9180980 0.6537396 0.4720378 0.55751177 0.6236419 0.8021578 1.0000000
4197 0.9637124 0.8407669 0.8905596 0.7483427 0.7759829 0.8127083 0.7151374 0.6154271 0.71512519 0.7736776 0.8646951 0.7332354
3643 0.5986460 0.7027969 0.2690678 0.4202418 0.7189885 0.8055690 0.4306184 1.0000000 0.04516544 0.7438391 0.7326686 0.5973885
8032 0.6557620 0.7860899 0.5870830 0.6387778 0.6403442 0.7234194 0.7020791 0.6743760 0.59843499 0.5689813 0.8951720 1.0000000
11277 0.6999176 0.6685247 0.7686899 0.8933950 0.7956939 0.5969833 0.4933414 0.6816408 0.85372907 0.6013083 0.9520593 1.0000000
9832 0.8491552 1.0000000 0.9215995 0.6252095 0.4294503 0.8835991 0.9195803 0.4807426 0.25379528 0.6725421 0.9520755 0.5320507
6332 0.7622787 0.7693434 0.7570448 0.5935742 0.6288960 0.5780101 0.3474384 0.6859659 0.73088477 0.5331535 1.0000000 0.9616916
9658 0.8279939 0.8113819 0.7558761 0.9517256 0.7308321 0.6209629 0.4752560 0.7324232 0.82211729 0.5797188 1.0000000 0.9639141
      AML_16  AML_17  AML_18  AML_19  AML_20  AML_21  AML_22  AML_23
7407 0.9219798 1.0000000 0.5627118 0.9182456 0.8654244 0.9532464 0.9402126 0.7330444
6520 1.0000000 0.8529564 0.4479271 0.7296461 0.7335773 0.6595703 0.7094700 0.8633671
8468 0.7591863 0.8268626 0.8257730 0.6089074 0.7582115 0.7580282 0.9017533 0.7681199
4197 0.8000445 1.0000000 0.7498705 0.8874702 0.8355399 0.7878400 0.9002706 0.8443875
3643 0.6044466 0.7750646 0.9521822 0.9325081 0.7595156 0.3930302 0.4473735 0.8501978
8032 0.7338404 0.9018745 0.5494804 0.6442590 0.7727167 0.8648436 0.6813118 0.6230011
11277 0.8614619 0.9826842 0.3099667 0.8020043 0.7643858 0.5554557 0.7537430 0.7117476
9832 0.8812008 0.7624028 0.9325334 0.8196914 0.8638415 0.9198072 0.8734509 0.8692303
6332 0.8919024 0.8021240 0.5619346 0.7151630 0.8414530 0.8505688 0.7318009 0.6167045
9658 0.8755628 0.8978103 0.3523421 0.7399059 0.7840514 0.5478130 0.7767179 0.6835639
```

將三組各 10 個 → 合併成 30 selected genes

```
selected_genes <- c(rownames(ALL_top_10_genes),
                    rownames(MLL_top_10_genes),
                    rownames(AML_top_10_genes))
```

檢查是否有重複挑選的基因

```
selected_genes <- unique(selected_genes)
selected_genes
length(selected_genes)
```

Output:

```
> selected_genes <- unique(selected_genes)
> selected_genes
[1] "11297" "1119" "2592" "1737" "11718" "3768" "6048" "8165" "5161" "9882" "7136"
[12] "7930" "2770" "11643" "1176" "7170" "10797" "11368" "8050" "5591" "7407" "6520"
[23] "8468" "4197" "3643" "8032" "11277" "9832" "6332" "9658"
> length(selected_genes)
[1] 30
```

4. Use SVM to classify (條件:training data 同時當 testing data)

安裝套件

```
BiocManager::install("e1071")
library("e1071")
```

訓練模型

```
train_data <- data_norm[selected_genes, ]  
x_train <- t(train_data)  
y_train <- factor(labels)  
svm_model <- svm(x_train, y_train)  
pred <- predict(svm_model, x_train)
```

5. Make confusion matrix

```
confusion_matrix <- table(Predicted = pred, Actual = y_train)  
confusion_matrix
```

Output:

```
> confusion_matrix  
      Actual  
Predicted ALL  AML  MLL  
      ALL   20   0   0  
      AML   0  20   0  
      MLL   0   0  17
```